

Reflexión

Al realizar esta actividad, se pide que un código sea capaz de leer un archivo de texto y ordenarlo. Yo utilice un ordenamiento mergesort, para ordenarlo de manera ascendente en un archivo nuevo ya ordenado, aunque en una parte del código se despliega de manera descendente; el nuevo archivo es ascendente para que se vea más limpio y el despliegue descendente es debido a requerimientos de la rúbrica de diseño.

Para este problema se consideraron 5 algoritmos: bubble sort, insertion sort, selection sort, mergesort y quicksort. Estos algoritmos tienen como propósito ordenar, aunque lo hacen de distintas maneras llegan al mismo resultado. El tiempo de ejecución es el que tarda dependiendo de qué algoritmo se use y de qué tan grande es el array o vector que vamos a ordenar. Esto se puede ver en la siguiente tabla.

Tabla de complejidad computacional:

Array Sorting Algorithms				
Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$
<u>Cubesort</u>	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$

- **Bubble Sort** consiste en intercambiar valores adyacentes de manera repetida, el valor de la izquierda se compara con el valor adyacente hasta que todos los valores están ordenados.

- **Insertion Sort** consiste en tomar el primer elemento como un elemento ordenado, luego se van comparando los valores de la izquierda para ver si el elemento debería estar antes en el array o ser considerado ordenado. Si el elemento debería ir antes, se inserta en la posición correspondiente.
- **Selection Sort** consiste en iteraciones que comparan el primer valor no ordenado con todos los valores que siguen en el array hasta encontrar un valor más pequeño y cambiarlo de lugar con el valor pequeño.
- **MergeSort** consiste en dividir el array en la mitad varias veces hasta convertir las divisiones en elementos individuales, luego se rearmen los grupos, de la misma manera que se dividieron, comparando los valores y ordenarlos de manera correcta hasta ordenar todo el array.
- **QuickSort** consiste en utilizar un pivote en un punto aleatorio (usualmente es el último elemento) del array, luego se comparan los elementos del array de manera que los que sean más pequeños que el pivote estén a la izquierda y los más grandes a la derecha, cuando estén ordenados el pivote queda ubicado en su posición correspondiente. Como este algoritmo es recursivo cada grupo de la derecha y la izquierda se ordena con el mismo método, hasta que todos los elementos están ordenados.

Los valores de la tabla anterior indican la complejidad temporal (cuanto tardan en ordenar) de cada algoritmo, de los 5 se eligió mergesort debido a que su complejidad en los mejores y peores casos es la más constante y no baja tanto el rendimiento. Quicksort no fue elegido porque si se le da un vector o array ordenado tarda demasiado en ordenarlo, en el caso de mergesort tarda siempre lo mismo.

Para este problema también se tenía a la disposición 2 algoritmos de búsqueda: búsqueda binaria (sólo funciona con array ordenados) la cual consiste en buscar en un array un valor, partiendo el array a la mitad, comparando si es mayor o menor y descartar la otra mitad donde no esté ubicado el valor, así hasta que se encuentre el valor; y la búsqueda secuencial que busca 1 por 1 desde el inicio hasta el final del array, que es menos eficiente que binario porque busca de 1 a 1 y no descarta valores.

Yo utilice búsqueda secuencial, ya que era más simple de implementar en el programa ya que buscaba rangos de valores y los desplegaba.

Mi programa permite ordenar con mergesort una bitácora, dándole peso a las fechas numéricas y textuales, para crear un nuevo archivo que lo entrega todo ordenado de manera ascendente

Referencias

Big-O Algorithm Complexity Cheat Sheet (Know Thy Complexities!) @ericdrowell. (s. f.).

Recuperado 18 de septiembre de 2026, de <https://www.bigocheatsheet.com/>

GeeksforGeeks. (2026, 20 enero). *Sorting algorithms*. GeeksforGeeks. Recuperado 18 de

septiembre de 2026, de <https://www.geeksforgeeks.org/dsa/sorting-algorithms/>